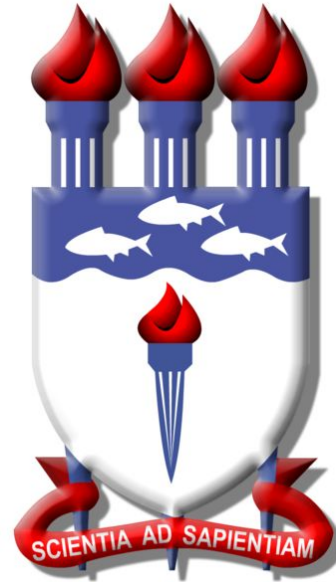




OTIMIZANDO FUNÇÕES DE *KERNEL* RACIONAIS PARA MÁQUINAS DE VETORES DE SUPORTE

Trabalho de Ciência de Dados

Aluno: Jose Augusto dos Santos Silva

Professor: Prof. Bruno Pimentel, Dr

Contextualização

- ❑ Aprendizado de Máquina se tornou um caminho necessário em diversas aplicações, incluindo PLN, visão computacional, medicina, finanças, etc;
- ❑ Máquinas de Vetores de Suporte (SVMs) são reconhecidas pela sua robustez e versatilidade em resolver problemas de classificação e regressão;
- ❑ SVMs têm como base a teoria do aprendizado estatístico ou teoria VC (Vapnik-Chervonenkis);
- ❑ Têm como objetivo encontrar o hiperplano ótimo que segrega os dados de classes distintas.

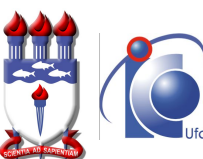
Motivação

- ❑ SVM na sua forma pura é um modelo linear;
- ❑ As Funções de Kernel fazem o papel de generalização em SVMs para problemas não lineares, frequentemente encontrado problemas reais;
- ❑ As aplicações reais dependem de uma boa função de kernel adaptada para os dados do problema em questão;
- ❑ Qual função de kernel utilizar?
- ❑ Qual o efeito da limpeza dos dados nos testes?

Lista de funções de kernel disponíveis

- Polinomial;
- RBF;
- Sigmoides;
- Fisher;
- Graph;
- String;
- Three;
- Path;
- Linear;
- Fourier;
- B-spline;
- Cosseno;
- Multiquadrática;
- Log;
- Cauchy;
- Tstudent;
- Thin plate;
- Harr;
- Daubechies;
- Coiflet;
- Symlet;
- Combinações lineares entre as próprias funções de kernel;
- Etc.

Fonte: ZHANG, J. A complete list of kernels used in support vector machines. **Biochem. Pharmacol.(Los Angel)**, v. 4, p. 2167-0501, 2015.



Morfologia das funções de kernel tradicionais

→ Linear:

$$K(x_i, x_j) = x_i \cdot x_j, \quad f(x) = x \quad \text{e} \quad g(x_i, x_j) = x_i \cdot x_j$$

→ Polinomial:

$$K(x_i, x_j) = (x_i \cdot x_j + r)^d, \quad f(x) = (x + r)^d \quad \text{e} \quad g(x_i, x_j) = x_i \cdot x_j$$

→ RBF:

$$K(x_i, x_j) = \exp(-\gamma \|x_j - x_i\|^2), \quad f(x) = \exp(-\gamma x^2) \quad \text{e} \quad g(x_i, x_j) = \|x_j - x_i\|$$

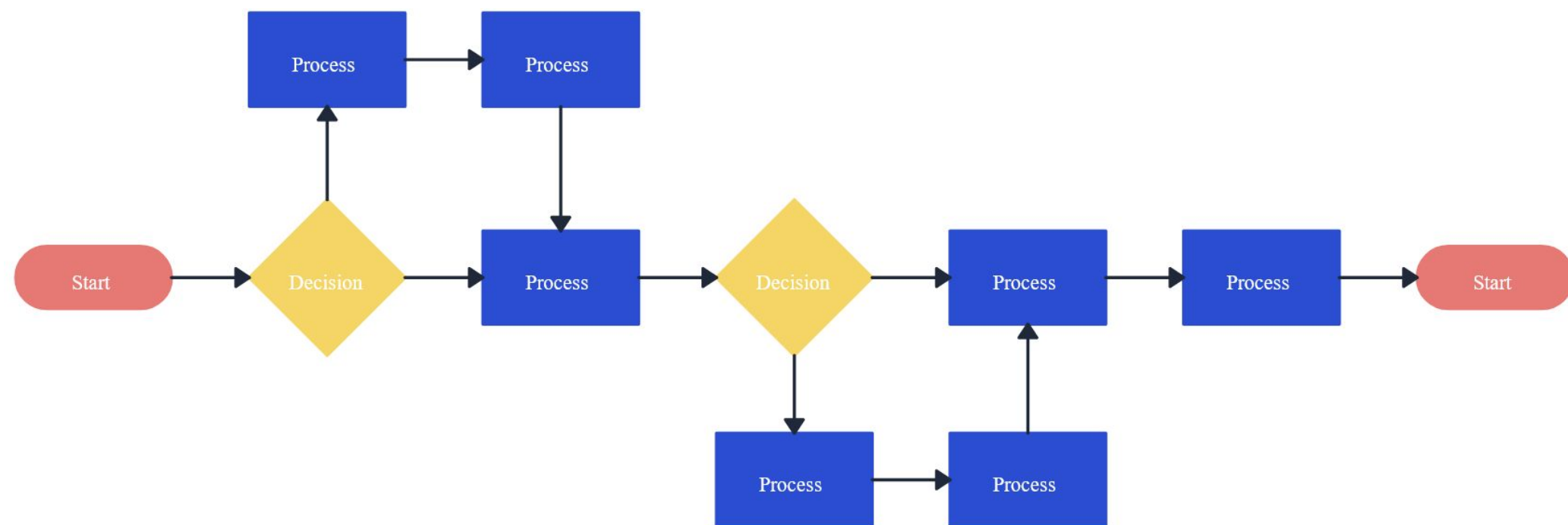
→ Sigmoidal:

$$K(x_i, x_j) = \tanh(\kappa x_i \cdot x_j + c), \quad f(x) = \tanh(\kappa x + c) \quad \text{e} \quad g(x_i, x_j) = x_i \cdot x_j$$

Morfologia: uma função real de uma variável (f) composta de uma função real de 2 variáveis (g)

Objetivo principal do projeto de Ciência de Dados

Adicionar limpeza de dados na metodologia de busca automática de funções de kernel para SVMs baseadas em funções racionais pela uso da DE na otimização de seus coeficientes



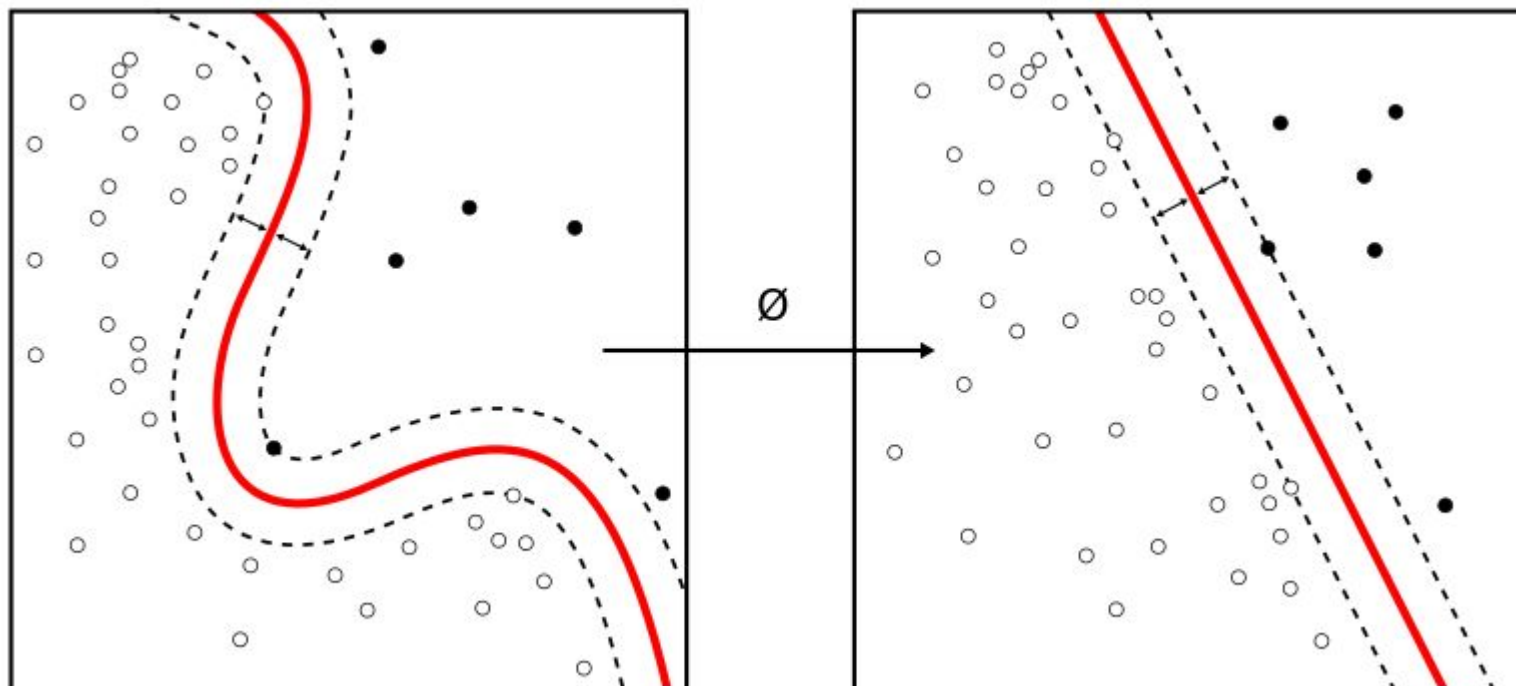
Fonte: <https://createely.com/>

Forma dual da SVM

Objetivo: encontrar um hiperplano que melhor separa os dados (maximiza a margem)

$$\text{maximize } W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j)$$

sob as restrições $0 \leq \alpha_i \leq C, \quad \sum_{i=1}^n \alpha_i y_i = 0$



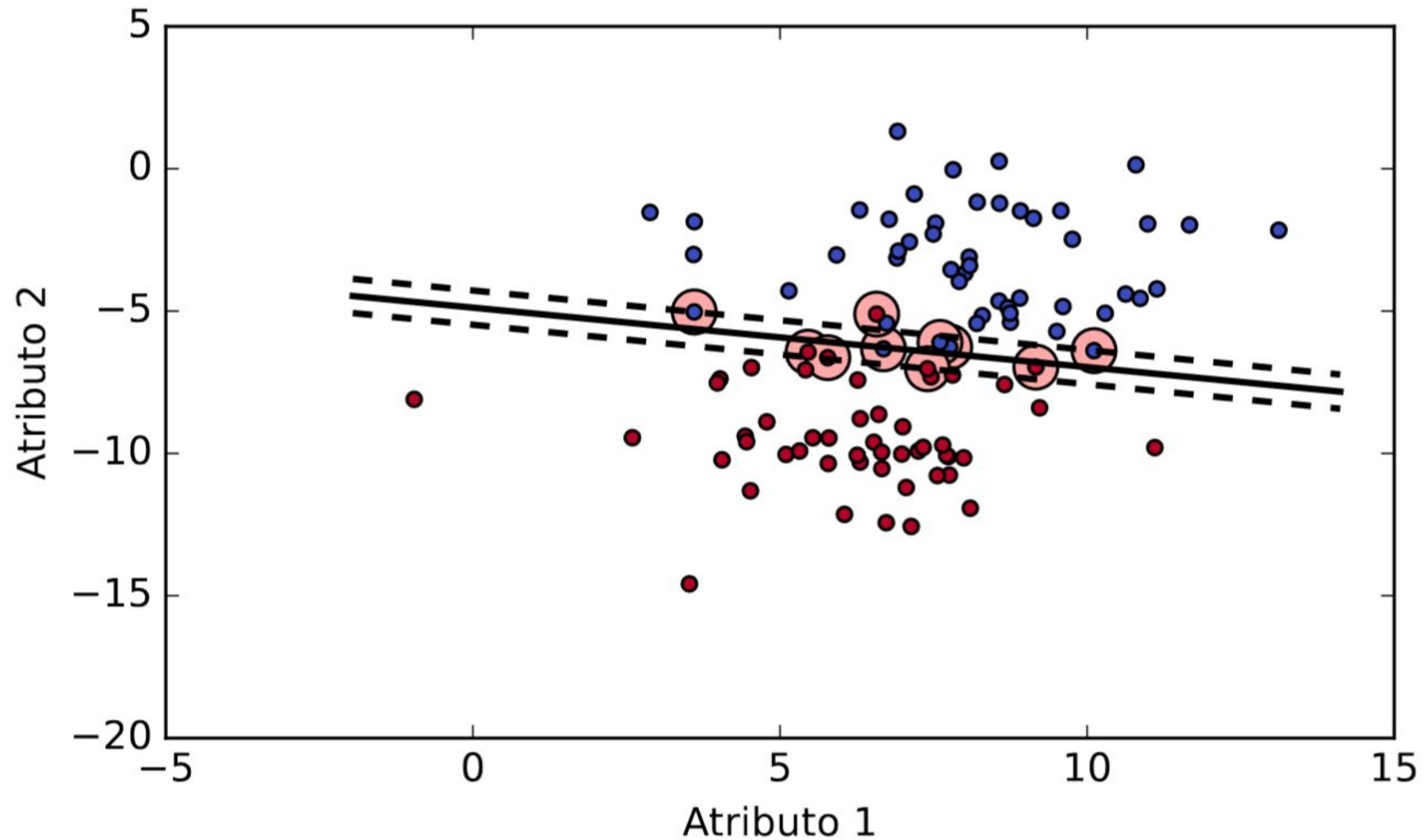
$$f(\mathbf{x}) = \sum_{(\alpha_i, x_i) \in SV} \alpha_i y_i (x_i \cdot \mathbf{x}) + b$$

$$f(\mathbf{x}) = \sum_{(\alpha_i, x_i) \in SV} \alpha_i y_i K(x_i, \mathbf{x}) + b$$

$$\hat{y} = \text{sign}(f(\mathbf{x})).$$

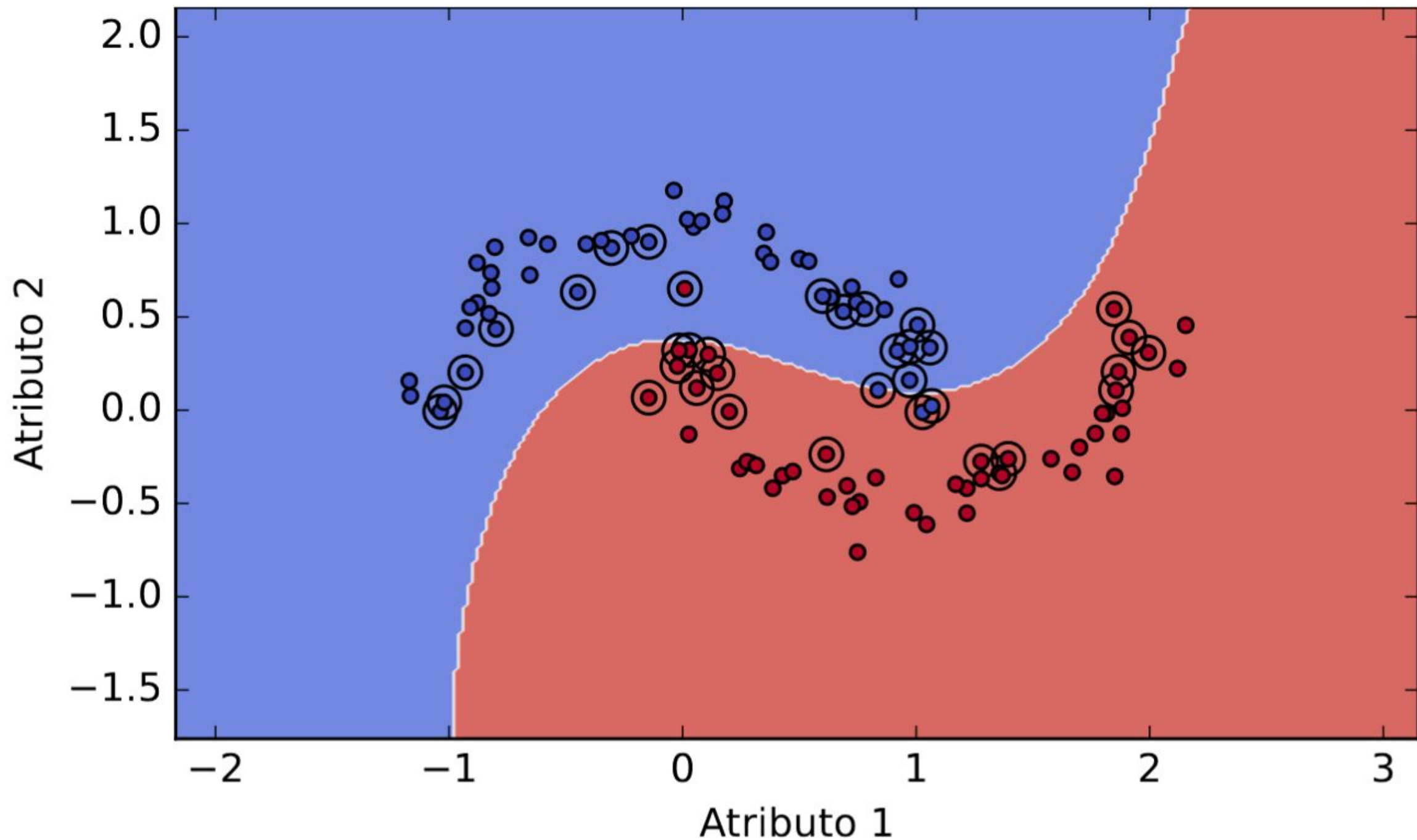
$$K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$$

Exemplo SVM linear com ajuste do C



Fonte: De autoria própria.

Exemplo SVM não linear (kernel) com ajuste do C



Fonte: De autoria própria.

Aproximação por funções racionais

$$R(x) = \frac{P(x)}{Q(x)} = \frac{a_n x^n + a_{n-1} x^{n-1} + \dots + a_1 x + a_0}{b_m x^m + b_{m-1} x^{m-1} + \dots + b_1 x + b_0}$$

- ❑ Características robustas de aproximação com uso em controle e processamento de sinais;
- ❑ Propriedades de convergência superiores em comparação a aproximações polinomiais
- ❑ Quando a função é conhecida, é possível aproximar com o método de Padé
- ❑ E quando não é conhecida?

Evolução Diferencial

- ❑ Algoritmo evolucionário para otimização de problemas multidimensionais de valor real;
- ❑ Utilizada em problemas não diferenciáveis, não lineares e multimodais;
- ❑ Operações matemáticas simples;
- ❑ Simples e fácil aplicabilidade.

Passo a passo Evolução Diferencial (“DE/rand/1/bin”)

1. Inicialize a população $\mathbb{P}$ com P vetores $[x_1, x_2, \dots, x_P]$ onde cada $x_i \in \mathbb{R}^d$;
2. Para cada geração n até N :
 - a) Para cada vetor x_i :
 - i. Selecione estocasticamente três vetores distintos a, b, c de $\mathbb{P}$;
 - ii. Crie um vetor mutante v utilizando: $v = a + F \times (b - c)$, onde F é um fator de escala;
 - iii. Crie um vetor de teste u calculado a recombinação binomial entre x_i e v ;
 - iv. Calcule a aptidão de u e x_i ;
 - v. Se u é melhor que x_i , substitua x_i por u ;
 - b) Atualize $\mathbb{P}$ baseado nas substituições feitas;
3. O melhor vetor na população final $\mathbb{P}$ é a solução.

Função de kernel representada por um indivíduo

1) Função $f(x)$:
$$f_I(x) = \frac{\sum_{i=1}^G I_i x^i}{\sum_{i=G}^{2G} I_i x^{G-i}}$$

2) Função $g(u, v)$:
$$g(u, v) = \sqrt{\sum_{i=0}^d (u_i - v_i)^2}$$

3) Função de kernel como composição de f e g :
$$K_I(u, v) = (f_I \circ g)(u, v)$$

Métricas para avaliar o kernel do indivíduo

- 1) Taxa de verdadeiros positivos: $TVP = \frac{TP}{TP + FN}$
- 2) Taxa de verdadeiros negativos: $TVN = \frac{TN}{TN + FP}$
- 3) Acurácia balanceada: $BAS = \frac{1}{2}(TVP + TVN)$
- 4) Número normalizado de iterações para convergência: $ITR_n = \frac{ITR}{ITR_{max}}$
- 5) Número normalizado de vetores de suporte: $NSV_n = \frac{NSV}{|X_{tr}| \times \frac{k-1}{k}}$

Aptidão de um indivíduo e o problema de otimização

❑ Função de aptidão:

$$Aptidão(I) = -BAS + \frac{1}{\frac{|X_{tr}|}{k}} (ITR_n + NSV_n)$$

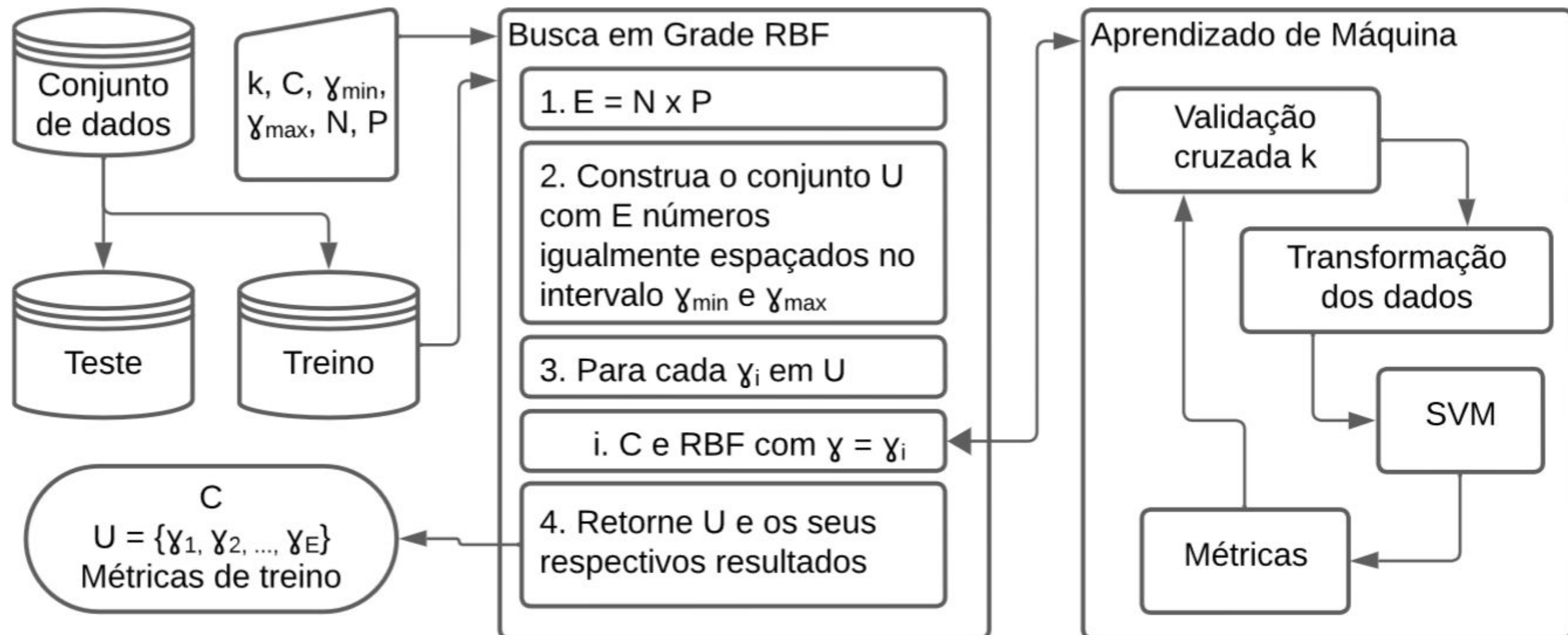
❑ Definição do problema:

$$\min Aptidão(I) = -BAS + \frac{1}{\frac{|X_{tr}|}{k}} (ITR_n + NSV_n)$$

sujeito a $-1 \leq I_i \leq +1, \quad i = 0, 1, \dots, 2G - 1$

Metodologia experimental para o kernel RBF

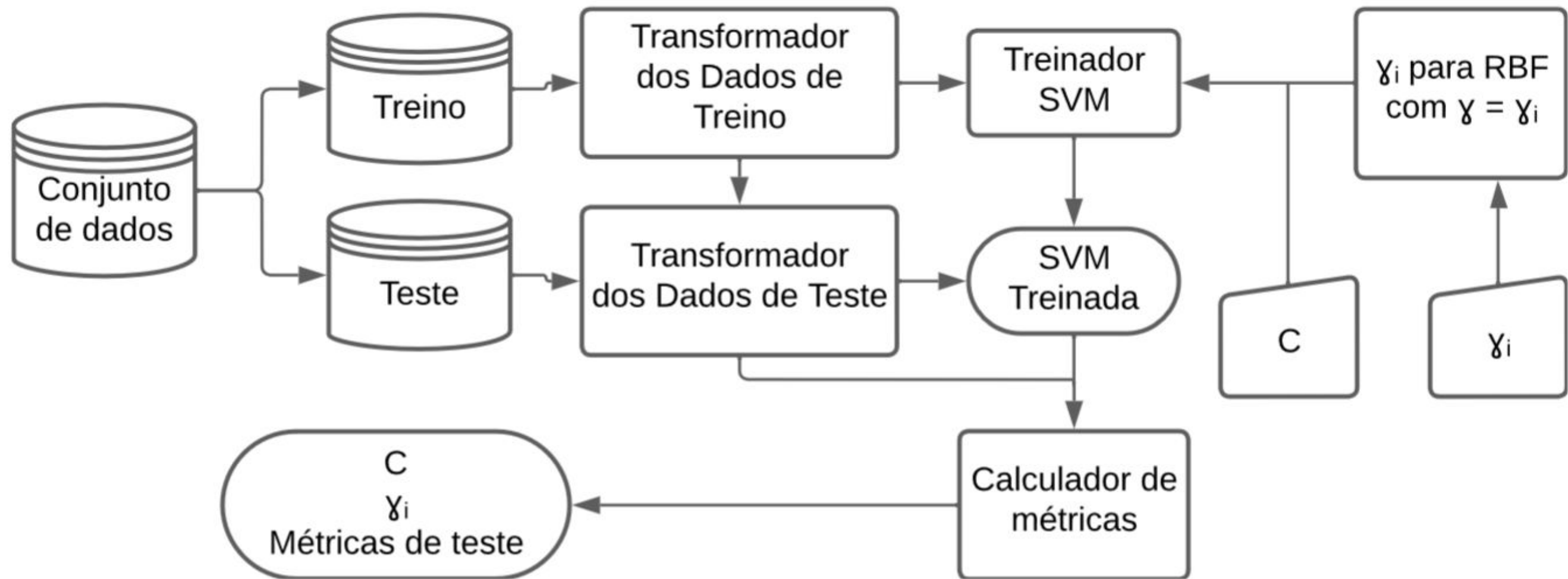
Figura 5 – Fluxo de treino para o *kernel* RBF.



Fonte: De autoria própria.

Metodologia experimental para o kernel RBF

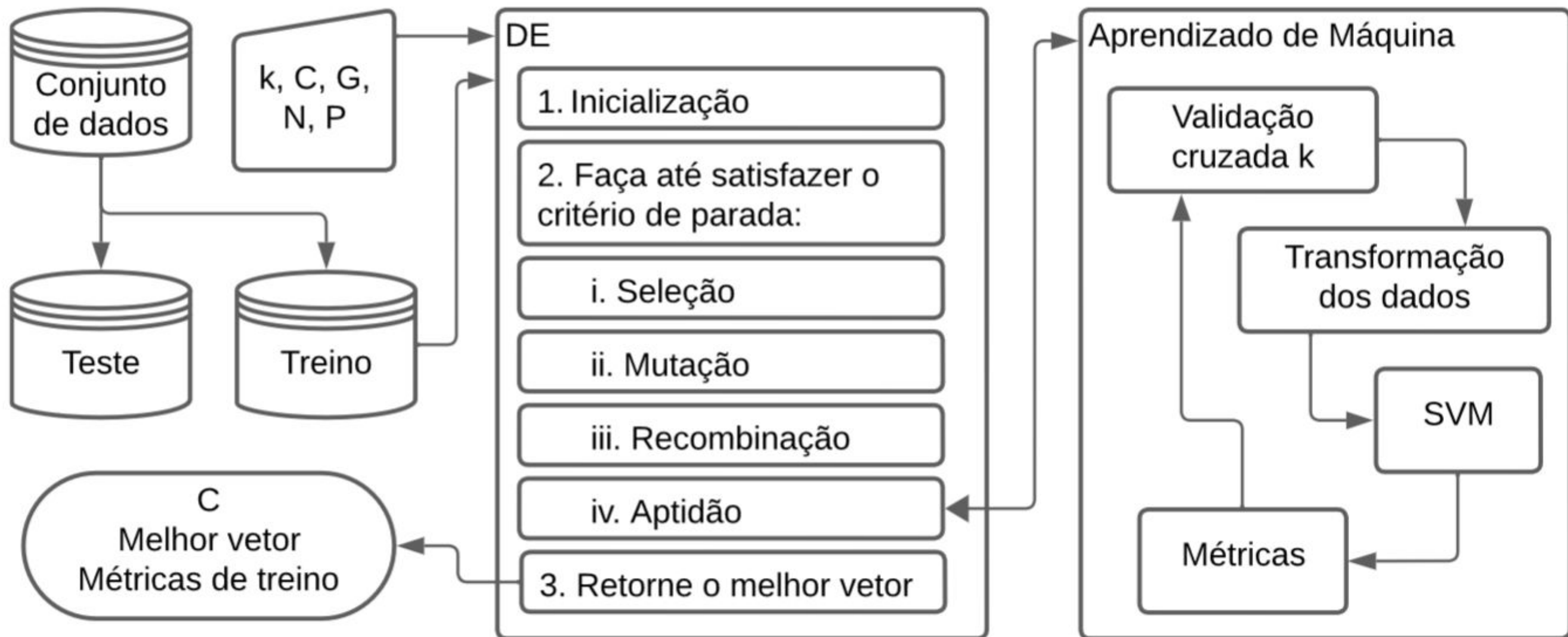
Figura 6 – Fluxo de teste para o *kernel* RBF.



Fonte: De autoria própria.

Metodologia experimental para o kernel racional

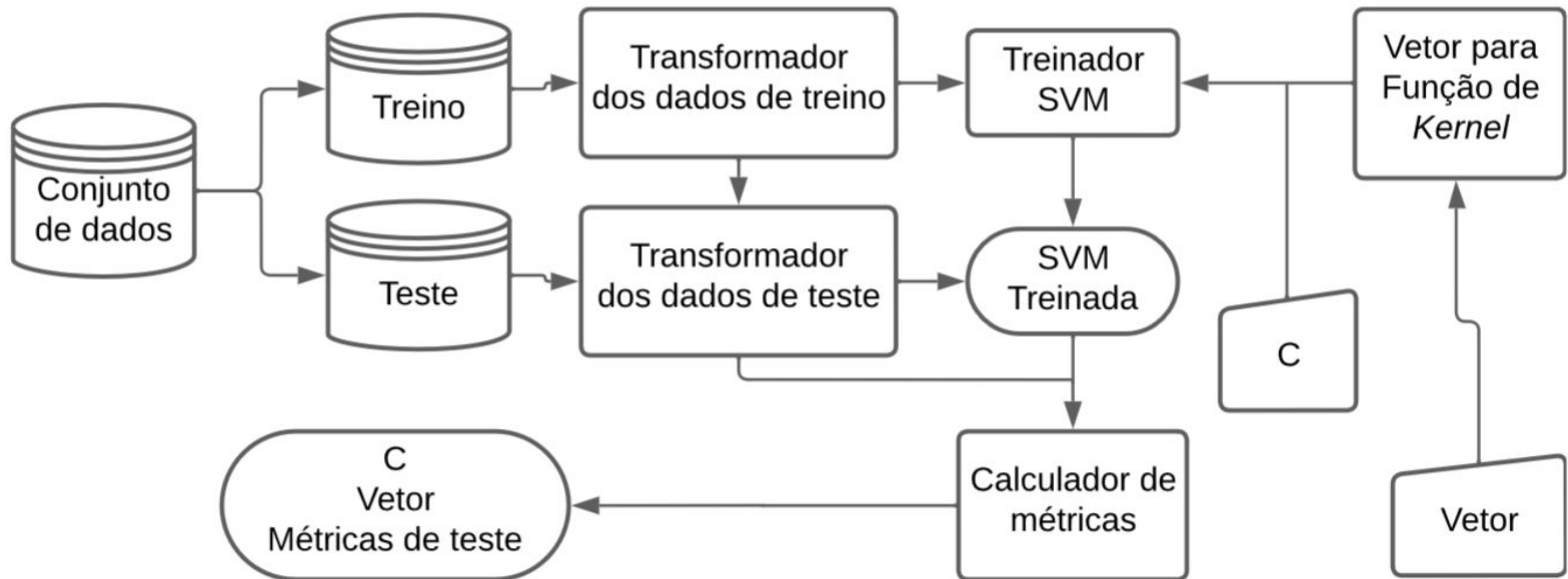
Figura 7 – Fluxo de treino para o *kernel* proposto.



Fonte: De autoria própria.

Metodologia experimental para o kernel racional

Figura 8 – Fluxo de teste para o *kernel* proposto.



Fonte: De autoria própria.

Resultados do TCC sem limpeza dos dados

Tabela 2 – Cinco melhores configurações para o *kernel* RBF

Posição	C	γ	BAS^{tr}	ITR_n^{tr}	NSV_n^{tr}	BAS^{te}	ITR_n^{te}	NSV_n^{te}
1º	2048	0,000108	0,724423	0,807496	0,524519	0,717407	0,895833	0,524349
2º	2048	0,000094	0,724386	0,741244	0,525258	0,716849	0,837556	0,524919
3º	2048	0,000096	0,724326	0,758091	0,525123	0,717157	0,831139	0,524837
4º	2048	0,000098	0,724325	0,764593	0,524926	0,716991	0,840926	0,524864
5º	512	0,000461	0,724320	0,788019	0,524817	0,716343	0,895454	0,525000

Fonte: De autoria própria

Tabela 3 – Cinco melhores configurações para o *kernel* racional

Posição	C	G	F	BAS^{tr}	ITR_n^{tr}	NSV_n^{tr}	BAS^{te}	ITR_n^{te}	NSV_n^{te}
1º	32	3	0,743240	0,748332	0,114598	0,510708	0,706577	0,142065	0,504207
2º	8	4	0,742657	0,748237	0,137626	0,547639	0,707015	0,169889	0,540798
3º	2048	3	0,742835	0,748059	0,130409	0,511128	0,705469	0,162685	0,506053
4º	32	4	0,742285	0,748039	0,170072	0,536544	0,702574	0,207565	0,530646
5º	128	3	0,742105	0,747352	0,126806	0,517509	0,708389	0,158657	0,516069

Fonte: De autoria própria

Parâmetros experimentais

- Conjunto de dados “Pima”
- Kernel racional com regularização C e número de coeficientes G em $\{(), (), (), (), ()\}$;
- SVM com $ITR_{max} = 1800$;
- *Holdout*: 80% dos dados para treino e 20% para teste;
- Validação cruzada de $k = 4$ nos dados de treino;
- DE com taxa de recombinação $CR = 0,9$;
- DE com fator de escala $F = 0,8$;
- DE com tamanho da população $P = 30$
- DE com número de gerações $N = 40$
- Execução de 30 experimentos para cada combinação.

Resultados sem e com a limpeza dos dados

C	G	F	BAS^{tr}	ITR^{tr}	NSV^{tr}	BAS^{te}	ITR^{te}	NSV^{te}
128.0	3	0.7391	0.7432	0.1224	0.5152	0.7180	0.1500	0.4943
8.0	4	0.7390	0.7430	0.1202	0.5045	0.7156	0.1570	0.4928
2048.0	3	0.7385	0.7421	0.0811	0.4649	0.7181	0.1033	0.4485
32.0	3	0.7381	0.7418	0.0870	0.4777	0.7124	0.1166	0.4712
32.0	4	0.7375	0.7416	0.1229	0.5166	0.7142	0.1726	0.5044

Table 1. Resultados para o conjunto Pima sem a limpeza dos zeros

C	G	F	BAS^{tr}	ITR^{tr}	NSV^{tr}	BAS^{te}	ITR^{te}	NSV^{te}
8.0	4	0.7600	0.7649	0.1748	0.5697	0.7435	0.2356	0.5553
32.0	4	0.7583	0.7637	0.2306	0.5973	0.7389	0.3110	0.5884
32.0	3	0.7593	0.7635	0.1350	0.5167	0.7430	0.1796	0.5095
2048.0	3	0.7577	0.7625	0.1799	0.5626	0.7398	0.2404	0.5560
128.0	3	0.7566	0.7617	0.1979	0.5803	0.7373	0.2631	0.5729

Table 2. Resultados para o conjunto Pima com a limpeza dos zeros

Conclusões

- I. Potencial das funções racionais somados com a limpeza de dados de atingir resultados mais robustos na assertividade;
- II. Assertividade em treino e teste maiores em relação aos resultados do TCC;
- III. Necessidade de um número de iterações maior no estudo de caso;
- IV. Necessidade de uma quantidade maior de vetores de suporte no estudo de caso.

Limitações e trabalhos futuros

Limitações:

- Kernel válido e as condições de Mercer;
- Distância euclidiana;
- Estudo somente em um conjunto de dados.

Trabalhos futuros:

- Utilizar as condições de Mercer como restrições da otimização;
- Avaliar outras distâncias, ex.: Minkowski com pesos;
- Melhorias na função de aptidão
- Testar a proposta em vários conjuntos de dados

